HONEYWELL SOFTWARE ENGINEERING TEAMS

AI Champions Programme

Module 02

GitHub Copilot Enterprise Features and Engineering Workflows

Building a strong working knowledge of Copilot's engineering capabilities — chat, inline assistance, agent mode, and repository-aware workflows — as the practical foundation every later module builds on.

DURATION
2 Hours

FORMAT
Hands-On Lab

DELIVERED FOR
Honeywell Eng. Teams

How We'll Spend These Two Hours

Six connected moves — from the full capability map to a hands-on speed-and-quality comparison.

01

The Copilot Enterprise Capability Map

15 MIN

A guided tour of every capability this programme will draw on, grouped into four categories.

02

Chat, Inline Assist & Code Explanation

15 MIN

The everyday, conversational entry points into Copilot — where most engineers start.

03

Refactoring, Test Generation & Debugging

20 MIN

Using Copilot across a realistic code-quality workflow, not just first-draft generation.

04

Agent Mode & Repository-Aware Assistance

20 MIN

How agent mode reasons across multiple files, dependencies, and existing repository context.

05

Model/Task Selection & Enterprise Workflows

15 MIN

Choosing the right model and task mode, GitHub.com and terminal/CLI workflows, and safe usage practices.

06

Hands-On Lab: Manual vs. Copilot-Assisted

35 MIN

Perform the same engineering tasks with different capabilities and compare speed and quality.

Module 02 in the 12-Module Journey

Copilot fluency is the practical tool foundation every later engineering method is built on.

Why this matters: Module 03 recaps prompt engineering and moves into context engineering; Module 04 builds spec-driven development on top of the same Copilot capabilities practiced here. Fluency now means less friction later.

GitHub Copilot Enterprise: The Capability Map

Four categories of capability — everything this module and later modules will draw on.

CONVERSATIONAL & INLINE	CODE QUALITY	AGENT & REPOSITORY	ENTERPRISE WORKFLOW
Chat	Refactoring	Agent Mode	GitHub.com Workflows
Inline Code Assistance	Test Generation	Repository-Aware Assistance	Terminal/CLI
Code Explanation	Debugging	Model/Task Selection	PR/Code Review
			Reusable Instructions & Skills

Chat, Inline Assist & Code Explanation

The conversational entry points most engineers reach for first.

Chat / Conversation

Ask questions, request changes, and reason about code in natural language, scoped to the open workspace or repository.

Inline Code Assistance

Suggestions generated directly in the editor as you type, for completions, boilerplate, and small edits.

Code Explanation

Walk through unfamiliar or legacy code — especially useful before making any brownfield change.

Beyond First-Draft: The Code-Quality Workflow

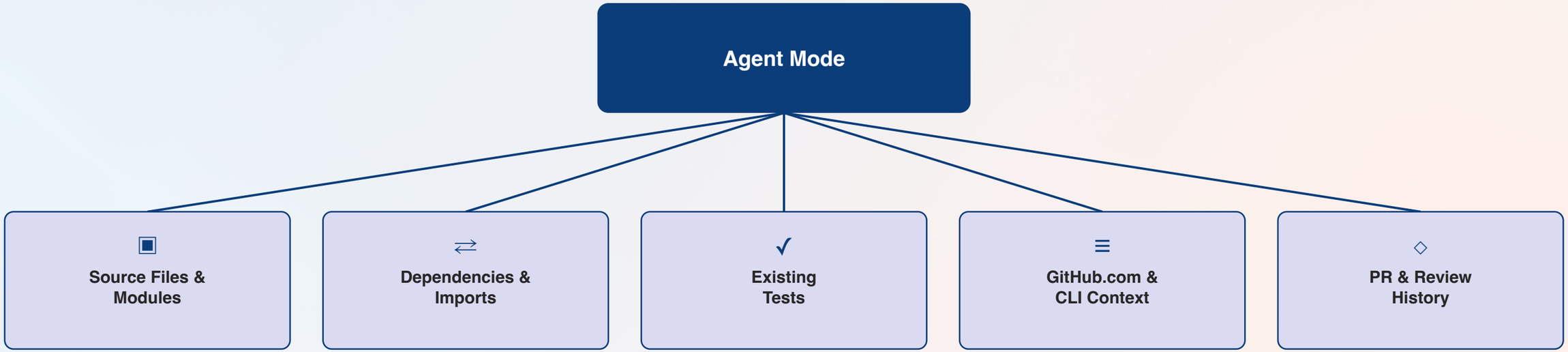
Copilot applied across the moves that come after the first version of the code exists.

Generate	Refactor	Generate Tests	Debug
Produce a first working version of a function, endpoint, or component from a natural-language description.	Improve structure, naming, and readability without changing behavior — with Copilot proposing the diff.	Create unit tests covering the new or refactored code, including edge cases the author may not think of.	Diagnose a failing test or reported defect, with Copilot reasoning from the stack trace and surrounding code.

The throughline: Each stage is compared for speed and quality in this module's hands-on lab — the same discipline the programme later applies to full specification-driven features.

Agent Mode: Where Copilot Touches Your Repository

Repository-aware assistance reasons across more than the file currently open.



Why this matters: This is the same repository-aware reasoning that Module 05's brownfield workflow and Module 07's MCP-enabled agents build on — agent mode is the foundation, not a separate topic.

Model/Task Selection & Safe Enterprise Usage

Choosing the right mode for the task, and the habits that keep Copilot usage governed.

NOT THAT	DO THIS
Use one model/mode for every task, regardless of fit	Match model/task selection to the task's size and risk
Let agent mode run unattended on repository-wide changes	Review agent-mode diffs before accepting, especially in brownfield code
Paste sensitive data into chat without checking policy	Follow enterprise data-handling policy for any prompt or context
Treat GitHub.com and CLI workflows as separate, ungoverned tools	Use GitHub.com and terminal/CLI workflows consistently with team conventions

Hands-On Lab Preview: Manual vs. Copilot-Assisted

The 35-minute activity that closes this module — the same task, worked both ways.

Task	Manual Approach	Copilot-Assisted Approach
Generate / explain / refactor code	Write and reason through it unaided	Chat, inline assist, and code explanation
Create tests	Hand-write unit tests from scratch	Generate tests, then review for coverage gaps
Debug a defect	Trace the stack manually	Debug with Copilot reasoning from the failure
Repository-wide task	Search and edit files one by one	Agent mode, reviewed before accepting

Module 02 Outcomes & What's Next

- The full Copilot Enterprise capability map is understood
- Chat, inline assist, and code explanation have been used hands-on
- Refactoring, test generation, and debugging workflows are practiced
- Agent mode's repository-aware reasoning is understood
- Model/task selection and safe enterprise usage practices are clear
- Manual and Copilot-assisted approaches have been compared for speed and quality

NEXT MODULE

Module 03 — Prompt Engineering Recap and Context Engineering (2 hours). A practical recap of prompt engineering, then a move into selecting, structuring, and managing context to avoid token inflation.

Questions & Discussion

Module 02 — GitHub Copilot Enterprise Features and Engineering Workflows